

Virtualizer Usage Notes

Project 1 Experimental Steps

Article I. Article I. TLM Creator

Section 1.01 Prepare The Implemented SystemC Module

Assume the completed SystemC module is named `mName`, then

- `mName.h` must use `#ifndef` to avoid repeated includes. If it is not added, the Virtualizer compiler will report an error. Add `#ifndef` as follows:

```
#ifndef SYSTEMC_INCLUDE_MNAME_H
#define SYSTEMC_INCLUDE_MNAME_H
...
#endif /* SYSTEMC_INCLUDE_MNAME_H */
```

- **Modify `SC_MODULE`, changing `SC_MODULE(mName) {...}` to**
`class mName: public sc_module{...}`
- **Add `public` encapsulation on the first line of the `class` definition above**

```
class mName: public sc_module {
public:
    ...
};
```

The C++ version of Virtualizer does not have default `public` encapsulation, so `public:` must be added for compilation to succeed.

- **Modify `SC_CTOR`, changing `SC_CTOR(mName) {...}` to the C++ constructor style as follows**

```
mName(sc_module_name mName)
```

- If the constructor has additional parameters, use normal C++ syntax to initialize them. Assume `mName` has a `BASE` address parameter, plus one TLM2 initiator

port Msock and one TLM2 target port Ssock:

```
class mName: public sc_module {
public:
    tlm_utils::simple_initiator_socket<mName> Msock;
    tlm_utils::simple_target_sock<mName> Ssock;
    sc_dt::uint64 BASE;
    ...
    mName(sc_module_name mName, sc_dt::uint64 base_):
        Msock("Master"), Ssock("Slave"), BASE(base_) {
        ...
    }
};
```

- Normally, `mName.cpp` does not need to be modified. However, the Virtualizer compiler is very picky. If it finds a problem, you can only fix it to import the module successfully. For example, if a `sc_dt::uint64` variable is compared with `> 0x0` or `>= 0x0`, because an unsigned int is always greater than 0, it will be regarded as unreasonable, and a compilation error will occur, so it must be corrected.

Virtualizer seems to support only TLM2 simple sockets and not multi-passthrough sockets. At present, how to use multi-passthrough sockets in Virtualizer remains unclear, so initiator and target ports have to be declared individually using `simple_initiator_socket` and `simple_target_socket`.

Section 1.02 Import SystemC Module

This section explains the steps for creating the completed SystemC module in Virtualizer. The order of execution for the following steps is very important. If you do not follow the order, it will be very troublesome to handle, and it would be better to simply right-click the project name in the `Project Explorer` window and choose `Delete` to kill it and start over:

1. Execute

New TLM Project -> TLM Starting Point + SystemC Import

2. The Project name and module name must be the same. Let us use PJ1Bus0 as an example for the following steps. When the Finish button lights up, click Finish.
3. In the PJ1Bus0.tlmc window, choose the Settings tab, then expand the General tab down below. In Model Information, give the Vendor field a distinctive name. For example, I use "ESL Class, EE NCKU." This step will add that distinctive name to the component list, making it easier to find later when adding it to a VDK Project.
4. In Project Explorer, expand PJ1Bus0, move the mouse to SystemC/include, right-click, and choose New->Header File, enter the file name PJ1Bus0.h, and click Finish.
5. PJ1Bus0.h is generated, and #ifndef/#endif is automatically added. You can copy the implemented SystemC .h content into it in the Virtualizer editor, or use an external editor (you do not need to close Virtualizer). When saving with an external editor, the editor in Virtualizer will update the content automatically.
6. Then in Project Explorer, move the mouse to SystemC/src, right-click and choose New->Source File, enter PJ1Bus0.cpp as the file name, and click Finish.
7. PJ1Bus0.cpp is generated. Likewise, you can copy the implemented SystemC .cpp content into it in the Virtualizer editor, or use an external editor (you do not need to close Virtualizer). When saving with an external editor, the editor in Virtualizer will automatically update the content.
8. Return to the PJ1Bus0.tlmc window, go to the Settings tab, then click the Build tab down below. In Compiler->Source Files and Header Files, you will see the newly added PJ1Bus0.h and PJ1Bus0.cpp. Open Compiler->Compile Flags and add -std=c++17.
9. Return to the Settings tab, then expand General and select SystemC Import. Leave the Model Instance field blank on the right-hand side and click Import directly. If the .h and .cpp files check out fine, a new Select SystemC Model window will appear. If there is anything in the three

Parameter tables, set the initial values; only then will the Finish button appear. Click Finish.

10. The above steps will automatically recognize TLM2 socket ports and `sc_in/sc_out` ports, and list them in the `Module.tlmc -> Interface` table. If there is no error, you can directly Build and Run. The Green bar will appear.
11. If the module has control registers (such as the eight control registers in DMA2), you must define them manually in `Module.tlmc -> Memory Map`. Virtualizer cannot identify them automatically.
12. If the module has interrupt pins (such as `INT0` and `INT1` in DMA2), you must change the Category of the interrupt pins to `Interrupt` in `Module.tlmc->Interface`.

Please note that after `DMA2` is imported, you must manually add the following eight control registers in `Memory Map`: `SOURCE0` (0x0, 4 bytes), `TARGET0` (0x4, 4 bytes), `SIZE0` (0x8, 4 bytes), `START0` (0xC, 4 bytes), `SOURCE1` (0x10, 4 bytes), `TARGET1` (0x14, 4 bytes), `SIZE1` (0x18, 4 bytes), and `START1` (0x1C, 4 bytes).

In addition, after `DMA2` is imported, the `base_under Parameters -> Constructor Parameters` is automatically recognized, and its initial value should be set to `0x10000000`.

Virtualizer verification: writing `ModuleTest.cc` under `Project Explorer -> Module -> Tests -> Unittests` is very troublesome, but it is close to the UVM concept. Although writing a SystemC `sc_main` testbench by hand would be much faster, using Virtualizer's verification method — especially combining `SCML2_TEST()` and `SCML2_ASSERT_THAT()` — is a thorough scheme for unit-test verification, so it should be used often. However, in the end, you must test on the virtual platform using software, where drivers and apps make a complete verification.

Section 1.03 Porting TLM Project

Here we describe the steps to port a TLM Project from one workspace to another.:

- 1 First, we need to complete a TLM Project, the DMA2 Project for example, in a workspace DualARMv8_1.
- 2 In another workspace, say, DualARMv8_2, click File -> Import, the Import window appears. Expand General, select Existing Projects into Workspace, and click Next >.
- 3 Click Select root directory if it is not selected. Then use the Browse button to locate the DMA2 project folder inside the DualARMv8_1 folder, then click Open.
- 4 Back to the Step 2 window, make sure the DMA2 project is listed in the Projects box and selected, and click Finish. You should find DMA2 in the Project Explorer. ■

Article II. VDK Creator & ARMv8 Starting Point

Section 2.01 VDK Project

In the current tests, it was found that a VDK project is divided into two parts:

The hardware generated by `New -> VDK Project`, whose project name matches the designed SoC, is listed in the `Project Explorer` window.

The `PlatformTest` project, which accompanies the `VDK Project`, is also listed in `Project Explorer`. The software running on the SoC is placed here.

No matter how many `VDK projects` are created in a single `Virtualizer Workspace`, the `PlatformTest` project is used. Since the software has not yet been run, it remains unclear whether renaming the `PlatformTest` project will disconnect it from the `VDK Project`. Therefore, it is recommended that, within a single workspace, although multiple TLM projects may exist, there be only one `VDK Project` to ensure a 1-to-1 relationship with the `PlatformTest` project.

Section 2.02 ARM9 Project

Because ARMv8/ARMv9 are too complex, they are not suitable for small SoCs. Small embedded SoCs still often use ARM9, ARM8, or even ARM7. Virtualizer supports the three ARM processor types above, but the IP library we obtained does not seem to support simple AHB/AXI buses, and crossbar buses such as CCN and CCI are too complex for ARM7, ARM8, and ARM9. For the current experiments, using a self-made TLM2 bus is a feasible method. Although we have not yet tested the running software, it is possible to complete the `VDK project` and compile without errors. Please refer to the two TLM2 modules `PJ1Bus0` and `PJ1Bus1` implemented for Project 1.

Section 2.03 ARMv8 Starting Point

As described in **Error! Reference source not found.**, regardless of how many VDK

projects are created in each workspace, only one PlatformTest project is automatically created by Virtualizer for embedded software and testing. Therefore, it is recommended that each VDK project use its own workspace. However, it is still possible to create a bare-metal VDK Project without generating the PlatformTest project. And we will use that for Project 1.

Next, please allow me to describe the currently known best procedure to do Project 1:

- 1 Create a new folder as the Virtualizer workspace, namely, DualA35:

```
mkdir DualA35
```
- 2 Run Virtualizer as

```
vs -d DualA35 &
```
- 3 Since DualA35 is a newly created, empty workspace, the Virtualizer Welcome perspective should appear. If so, click New Project under VDK Creation. Otherwise, can click File -> New -> VDK Project.
- 4 In the New VDK Project window, give the Project name as DualA35. We want to make the Project name match the workspace folder name for better management. Then, in the Project type select VDK Example. And in the Template, fill in A35 to search for Arm Cortex-A35 Example under Arm FM QuickRef Examples -> CortexA_v8. Select it and click Finish. The DualA35 project is created, listed in the Project Explorer, and shown in the VDK Creation perspective.
- 5 Check the Design Hierarchy, you will find a Cortex-A35 named ARM0, 2 memories named MEM and MEM2, a SharedMemoryMap, a Bridge, and 4 UARTs named UART_0/UART_PHY_0 thru UART_3/UART_PHY_3.
- 6 Build and Run the project, it changes into the VDK Debug perspective and holds at an initial_crunch break point. You can check the lower-right window with tabs for Simulation Output, Console, Details, UART0, UART1, UART2, and UART3. Click Simulation Output to view the output, and you will find 4 CPUs. At this point, viewing UART0 through UART3, you find blank terminals.
- 7 Click the Resume button to resume the simulation, and you will see all 4 UART terminals show Dhrystone testbench results. You can find the Dhrystone executable

AXF files in the Parameters window of ARM0, under Imageinfo. With the path shown, you can find these files inside the DualA35 workspace.

- 8 Stop the simulation and switch back to the VDK Creation perspective. In the Design Hierarchy, select the ARM0, then select Parameters in the /ARM0 window. Pull it down to the bottom, you will find number_of_cpu with Design Value and Runtime Value both of 4. Change them both to 2. Save the project and repeat Steps 6 and 7. At this time, only UART0 and UART1 show Dhrystone results.
- 9 Import DMA2, PJ1Bus0, and PJ1Bus1 modules using the procedure described in Section 1.02 and/or Section 1.03. We need only DMA2 for this procedure. However, you are welcome to use PJ1Bus0 and PJ1Bus1 should you like to do it the other way.
- 10 Next, add DMA2 and 2 Memory_Generic (MEM) into DualA35, make necessary connections to construct the architecture requested by Project 1. At each step of the change, build and run the project to ensure the Dhrystone program still runs on cpu0/UART0 and cpu1/UART1. Notice that the memory map of Project 1 has been modified and re-uploaded to Moodle.

The *ARM Cortex A35 TLM2 LT PSP* manual has been uploaded to Moodle for your reference. The software part is not included in this document; please work it out, thanks.